

Tài liệu thuật toán

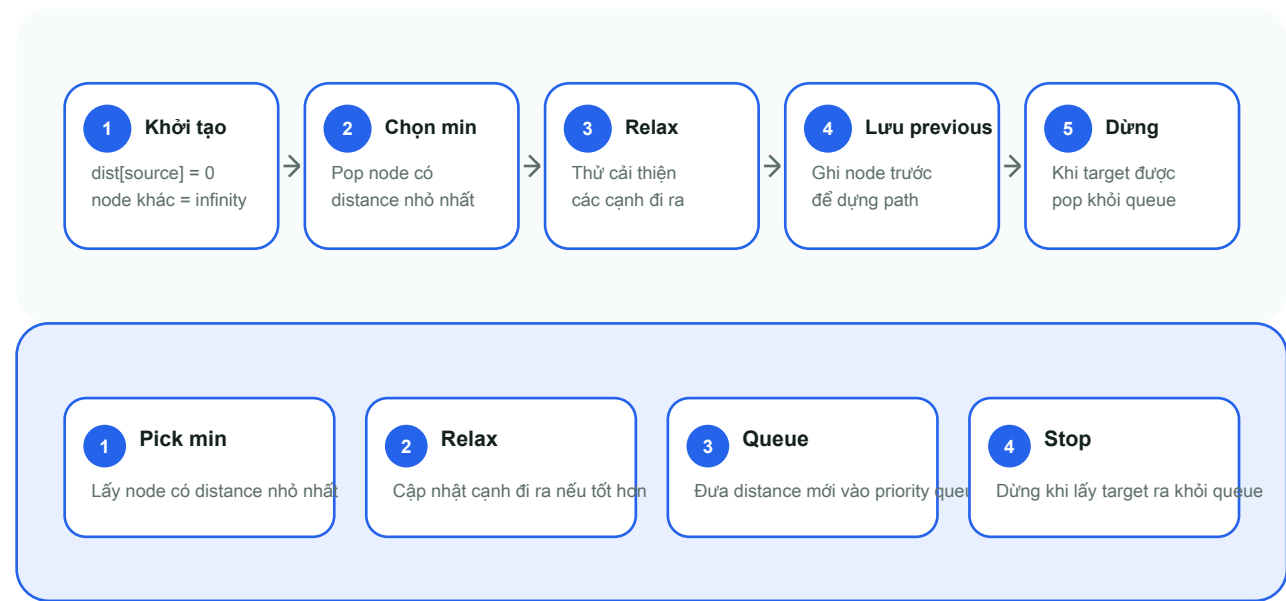
Tài liệu riêng giải thích cách Dijkstra tìm đường ngắn nhất trên graph có trọng số không âm.

Dijkstra Shortest Path

Dự án	RouteLab Group 1 - Shortest Path Visualizer
Mục tiêu	Giải thích thuật toán, contract dữ liệu, pseudocode, trace demo và checklist kiểm thử.
Dataset minh họa	Ho Chi Minh City graph, 7 nodes, source = 1, target = 6

1. Ý Tưởng Chính

Dijkstra giải bài toán đường đi ngắn nhất từ một nguồn đến các node còn lại. Trong demo RouteLab, ta dừng sớm khi target được lấy ra khỏi priority queue vì lúc đó distance của target đã là tối ưu.



2. Input, Output Và API Contract

Thành phần	Ý nghĩa
source	ID node bắt đầu.
target	ID node cần đi tới.
nodes	Danh sách điểm trên graph, có id/name/lat/lng.
edges	Danh sách cạnh có from/to/weight; weight không âm.
path	Chuỗi node từ source đến target.
visitedOrder	Thứ tự node được lấy ra khỏi priority queue.
relaxedEdges	Các cạnh đã được relax để phục vụ animation/demo.

POST /api/solve/dijkstra

Request:

```
{
  "source": 1,
  "target": 6,
  "nodes": [{ "id": 1, "name": "Ben Thanh Market", "lat": 10.7725, "lng": 106.698 }],
  "edges": [{ "id": "e1-2", "from": 1, "to": 2, "weight": 2.5 }],
  "directed": false
}
```

Response:

```
{
  "path": [1, 2, 3, 6],
  "totalCost": 7.5,
  "runtimeMs": 0.18,
  "visitedOrder": [1, 2, 0, 5, 3, 6],
  "relaxedEdges": [{ "from": 1, "to": 2, "cumulativeCost": 2.5 }]
}
```

3. Pseudocode Và Cách Cài Đặt

```

dist = map with infinity for every node
previous = empty map
dist[source] = 0
queue.push(source, priority = 0)

while queue is not empty:
    current = queue.popMin()
    if current is already finalized:
        continue

    mark current as finalized
    if current == target:
        break

    for each edge current -> neighbor:
        nextCost = dist[current] + edge.weight
        if nextCost < dist[neighbor]:
            dist[neighbor] = nextCost
            previous[neighbor] = current
            queue.push(neighbor, priority = nextCost)

return reconstructPath(previous, source, target)

```

Các điểm cài đặt cần chú ý:

- Không chấp nhận edge weight âm vì Dijkstra sẽ mất tính đúng.
- Dùng min-heap để lấy node có distance nhỏ nhất nhanh hơn quét toàn bộ.
- previous map giúp dựng lại path cuối cùng từ target về source.
- visited/finalized set tránh xử lý lại node đã có distance tối ưu.

4. Trace Trên Dataset HCM 7 Nodes

Bước	Sự kiện	Cạnh	Cost tích lũy
1	Relax edge	1 → 2	2.5
2	Relax edge	1 → 0	3.4
3	Relax edge	1 → 5	4.6
4	Relax edge	2 → 5	3.8
5	Relax edge	2 → 3	4.7
6	Relax edge	5 → 4	7.8
7	Relax edge	3 → 4	7.4
8	Relax edge	3 → 6	7.5
Done	Visited: 1 → 2 → 0 → 5 → 3 → 4 → 6	Path: 1 → 2 → 3 → 6	7.5

Kết quả demo

Từ 1 (Ben Thanh Market) đến 6 (Saigon Zoo), Dijkstra tìm path 1 → 2 → 3 → 6 với totalCost = 7.5.

5. Độ Phức Tạp

Dại lượng	Phân tích
Thời gian	$O((V + E) \log V)$ với binary heap.
Bộ nhớ	$O(V + E)$ cho adjacency list, dist, previous, queue.
Tính đúng	Đúng với graph có trọng số không âm.
Tác động khi graph lớn	Duyệt nhiều node nếu không có heuristic hướng về target.

6. Checklist Kiểm Thử

- ✓ Trả path bắt đầu bằng source và kết thúc bằng target.
- ✓ totalCost khớp tổng weight của các cạnh trong path.
- ✓ Reject graph có weight âm hoặc edge trở tới node không tồn tại.
- ✓ Case source/target không nối được phải trả lỗi rõ ràng.
- ✓ So sánh với graph nhỏ có shortest path biết trước.